

Windows and Linux CPU Scheduling

CSCI 460

Adiesha Liyanage

The ideas behind each scheduler

- Windows: select the highest current thread priority. (with dynamic boosts)
- Linux
 - CFS: favor the smallest weighted virtual runtime.
 - EEVDF: select the earliest virtual deadline among eligible tasks.
- All three balance responsiveness with other scheduling goals.

Windows priority levels

Range	Meaning
16–31	Real-time priority range
1–15	Variable priority range
0	Reserved for the zero-page thread

Larger priority numbers receive preference.

The zero-page thread clears free memory pages when other threads need no CPU.

Kernel execution and system threads

- System threads have their own priorities, chosen for their work.
- Some run at high priorities.
 - System threads need not all run at 31.
- A system call enters kernel mode without automatically raising thread priority.
- Interrupt request levels (IRQLs) separately control interrupt urgency.
 - In Windows, **IRQLs (Interrupt Request Levels)** are used by the kernel to control **which interrupts, and kernel activities are allowed to preempt the currently executing code.**

Windows base priority

Process class	Thread level	Base priority
Normal	Below normal	7
Normal	Normal	8
Normal	Above normal	9
High	Normal	13

Windows selection and preemption

- Choose a ready thread at the highest current priority.
- At equal priority, distribute time slices using round robin.
- A newly ready higher-priority thread preempts a lower-priority thread.
- A quantum is a time slice.
 - Its length depends on system settings.

Windows worked example

One CPU, 4 ms quantum, fixed priorities, boosts disabled.

Time (ms)	Running thread	Reason
0–4	A, priority 8	First equal-priority turn
4–6	B, priority 8	Next turn
6–8	C, priority 10	Higher priority preempts B

At 8 ms, C blocks. A or B can resume under queue rules.

Windows dynamic priority boosts

- Input events and completed waits can temporarily raise a thread's dynamic priority.
- These boosts help a waking or interactive thread respond promptly.
- After each full-time slice (quantum) it consumes, the boosted priority is reduced by one level until it returns to the thread's base priority.
- Threads whose base priority is 16–31 are in the real-time range and do not receive these normal dynamic priority boosts.

Foreground applications and responsiveness

- Windows tracks the foreground window and the thread/process associated with it.
- Clicking a window or using *Alt + Tab* can change which application is in the foreground.
- Foreground activity and user input can receive temporary scheduling preference to improve responsiveness.
- Simply having a visible GUI does not cause every thread in the application to receive a higher priority.

Starvation prevention and its limits

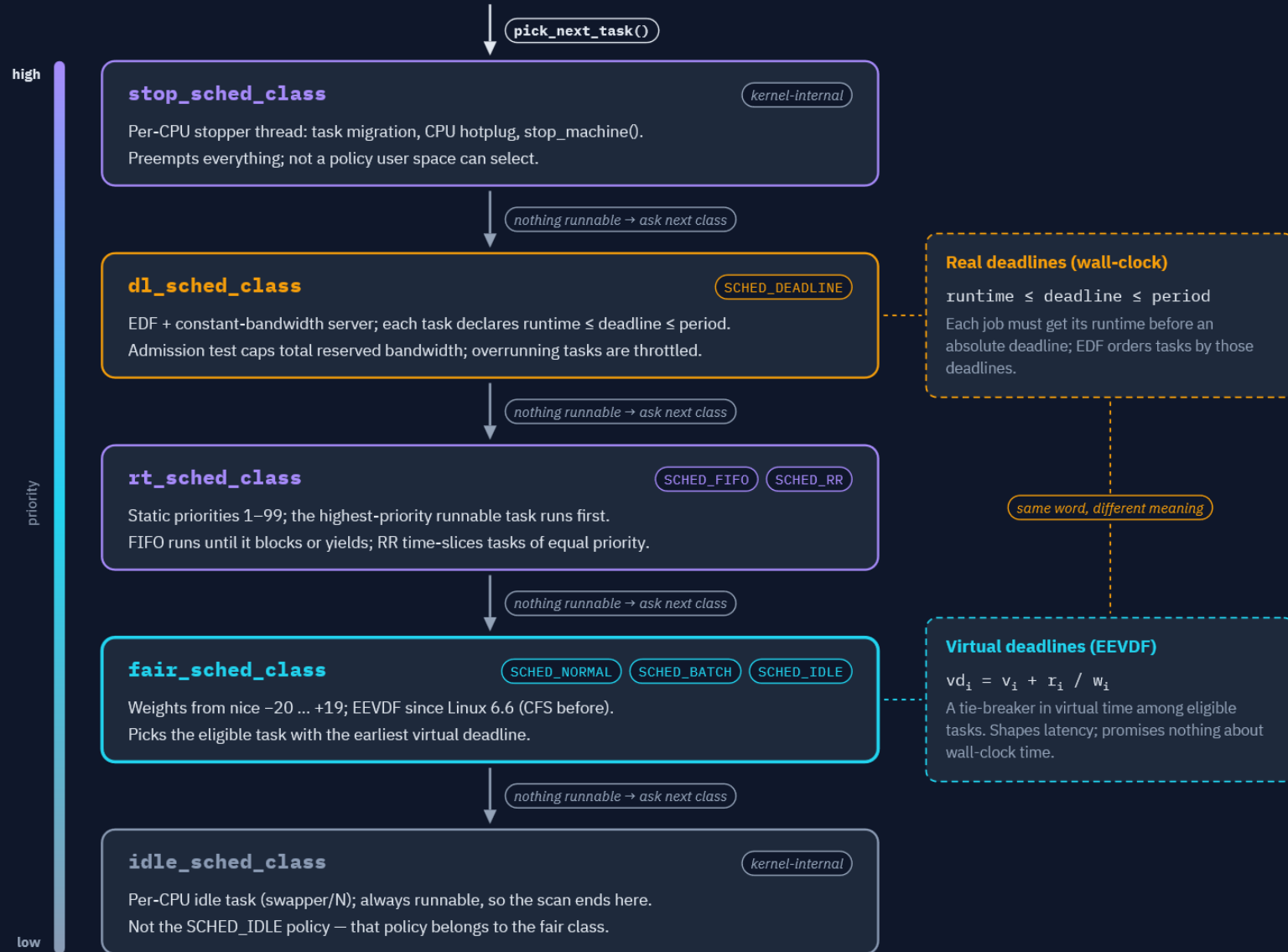
- Windows can temporarily boost ready threads that have waited too long for CPU time.
- The boost helps those threads compete with higher-priority runnable threads and reduces starvation risk.
- Temporary dynamic boosts later decay back toward the thread's base priority.
- Among threads at the same priority level, Windows uses round-robin time slicing.
- Continuously runnable real-time threads can still prevent ordinary-priority threads from running.

Linux scheduling policies

- Ordinary tasks normally use `SCHED_OTHER`, also called `SCHED_NORMAL`.
 - CFS and EEVDF provide selection within the fair scheduler.
- `SCHED_FIFO` and `SCHED_RR` provide real-time priority policies.
- `SCHED_DEADLINE` uses EDF with execution budgets and admission control.

Linux scheduling classes — priority order

The core scheduler asks each class in turn; the first class with a runnable task supplies the next task.



A runnable task in a higher class always preempts tasks in lower classes.

Caveat: RT bandwidth limits (default 950 ms of every 1 s; newer kernels enforce this with a deadline “fair server”) keep fair tasks from total starvation.

Linux weights and CPU entitlement

- A weight expresses a task's relative claim on CPU service.
- A lower nice value gives a task a larger scheduling weight, increasing its target CPU share when tasks compete.
- Informally: a lower nice value means the task is “less nice” to other tasks.
- For comparable runnable tasks, the target CPU share is approximately
 - $share_i = \frac{w_i}{\sum_j w_j}$ where w_i is task i 's weight.
- A task's actual share changes dynamically as competing tasks block, sleep, wake up, or become runnable.

Weighted CPU share example

One CPU, all tasks runnable, same fair group, no CPU quota.

Task	Weight	Target share	CPU in 30 ms
A	2	$2 / 4 = 50\%$	15 ms
B	1	$1 / 4 = 25\%$	7.5 ms
C	1	$1 / 4 = 25\%$	7.5 ms

If C blocks: A targets $\frac{2}{3}$ of CPU time, B targets $\frac{1}{3}$.

CFS virtual runtime

- Virtual runtime v records CPU service after adjusting for weight.
- Let Δt be actual CPU time, w the weight, and w_0 a reference weight.
- The increment is $\Delta v = \Delta t \times \frac{w_0}{w}$.
- A larger weight makes virtual runtime grow more slowly.

CFS worked selection example

Reference weight = 1. B runs for an illustrative 2 ms.

Task	Weight	Initial v	v after B runs for 2ms
A	2	6	$6 = 6 + 0 \times \frac{2}{1}$
B	1	4	$6 = 4 + 2 \times \frac{1}{1}$
C	1	5	$5 = 5 + 0 \times \frac{1}{1}$

First choice: B. Next choice in this model: C.

CFS fairness and implementation

- Classic CFS organizes runnable entities in a red-black tree by virtual runtime and favors the entity with the smallest virtual runtime.
- As a task runs, its virtual runtime increases, allowing other tasks to catch up.
- Granularity and wake-up placement balance fairness with responsiveness while avoiding excessive context switches.

EEVDF (Earliest eligible virtual deadline first)

- Lag measures CPU service owed relative to the fair allocation.
- Positive lag: the task has received less than its fair share.
- Negative lag: the task has received more than its fair share.
- A runnable task with nonnegative lag is eligible for selection.

EEVDF virtual deadlines

- Let V be the scheduler's **current virtual time**, representing the fair-service progress of the runnable set.
- Let v be the current virtual runtime, r the requested real CPU slice, w the task weight, and w_0 the reference weight.

$$d = v + r \times \frac{w_0}{w} \text{ (virtual deadline)}$$

- A task is considered only when it is **eligible** (it is owed fair CPU service)
- Among eligible tasks, EEVDF selects the task with the **smallest virtual deadline d** .
- The task's lag is determined by how far its service position is behind or ahead of V

EEVDF worked selection example

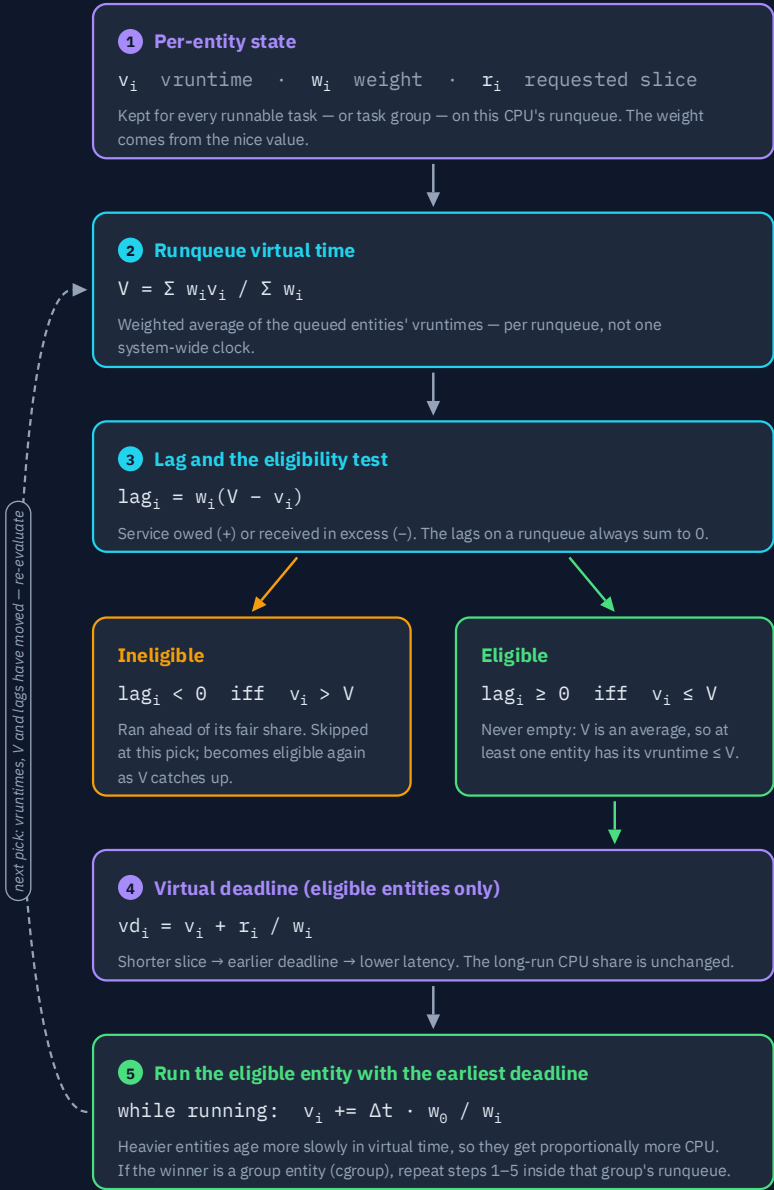
Equal weights. V is the runqueue virtual-time reference. $V = 10$.

Task	v	Slice r	Deadline d	Eligible?
A	8	8	16	Yes
B	9	5	14	Yes
C	13	0.5	13.5	No

EEVDF chooses B. C has the earliest deadline but is ineligible.

EEVDF — how the fair class picks the next task

Earliest Eligible Virtual Deadline First (Linux 6.6+). Runs inside `fair_sched_class`, separately on each CPU's runqueue.



EEVDF responsiveness and fairness

- A shorter request can give an eligible task an earlier virtual deadline.
- Eligibility prevents a task already ahead from always winning.
- When a request ends, accounting and the next request determine future choices.
- Linux began the CFS-to-EEVDF transition in version 6.6.

CFS and EEVDF compared

Property	Classic CFS	EEVDF
Main selection	Smallest virtual runtime	Earliest eligible deadline
Fairness accounting	Weighted virtual runtime	Weighted runtime and lag
Latency mechanism	Granularity and wakeups	Request length and deadlines
Fixed round robin?	No	No

Preemption and multiple CPUs

- Both Windows and Linux support preemptive thread scheduling.
- Kernel execution can temporarily disable or defer preemption while critical kernel work is in progress.
- CPU affinity restricts the set of processors on which a thread is allowed to run.
- Load balancing tries to distribute runnable work across CPUs while also considering cache locality, NUMA effects, and available CPU capacity.

Windows and Linux fair scheduling

Question	Windows	Linux fair class
Who runs next?	Highest dynamic priority	CFS v or EEVDF d
How is preference set?	Process class + thread level	Nice and group weights
Interactive response	Temporary boosts	Wakeups and fair accounting
Equal treatment	Round robin at same priority	Equal target CPU shares

Official references

Microsoft Learn

Scheduling Priorities, Priority Boosts, Context Switches, Multiple Processors

Linux kernel documentation

CFS Scheduler, EEVDF Scheduler, Scheduler Nice Design

Deadline Task Scheduling, Control Group v2

learn.microsoft.com/en-us/windows/win32/procthread/
docs.kernel.org/scheduler/